

คู่มือการใช้งาน Git & GitHub

สำหรับทีมพัฒนาโปรเจกต์ร่วมกัน

ครอบคลุม Database + Frontend + Backend (Python)

เนื้อหา: สร้าง Repo → Clone → VS Code → Branch → Push → Pull Request → .env

พร้อมคำอธิบายว่า ทำไม ต้องทำในแต่ละขั้นตอน

สารบัญ

- บทที่ 0 สิ่งที่ต้องเตรียมก่อนเริ่ม
- บทที่ 1 สร้าง Repository บน GitHub
- บทที่ 2 Clone โปรเจกต์มาที่เครื่อง (3 วิธี)
- บทที่ 3 เปิดโปรเจกต์ใน VS Code และเชื่อมต่อ Git
- บทที่ 4 สร้าง Branch ของตัวเอง (ทำไมห้ามทำงานบน main)
- บทที่ 5 สร้างไฟล์ HTML แล้ว commit & push ขึ้น branch ตัวเอง
- บทที่ 6 สร้างไฟล์ .env สำหรับรัน Python
- บทที่ 7 เปิด Pull Request เพื่อรวมงานกลับไป main
- บทที่ 8 อัปเดต branch ของตัวเองเมื่อเพื่อน merge งานเข้า main
- บทที่ 9 คำสั่ง Git ที่ใช้บ่อย (Cheat Sheet)
- บทที่ 10 ปัญหาที่พบบ่อยและวิธีแก้

บทที่ 0 สิ่งที่ต้องเตรียมก่อนเริ่ม

ก่อนจะเริ่มทำงานเป็นทีม ทุกคนในทีมต้องมีเครื่องมือพื้นฐานเหมือนกัน เพื่อให้คุยกันรู้เรื่อง ใช้คำสั่งเหมือนกัน และไม่มีปัญหา “เครื่องฉันรันได้ เครื่องเธอรันไม่ได้”

0.1 โปรแกรมที่ต้องติดตั้ง

1. Git — เครื่องมือควบคุมเวอร์ชัน (Version Control) ดาวน์โหลดที่ git-scm.com
2. VS Code — โปรแกรมแก้ไขโค้ด ดาวน์โหลดที่ code.visualstudio.com
3. Python (เวอร์ชัน 3.10 ขึ้นไป) — สำหรับฝั่ง backend ดาวน์โหลดที่ python.org
4. Node.js (ถ้าใช้ frontend framework เช่น React/Vue) — ดาวน์โหลดที่ nodejs.org
5. บัญชี GitHub — สมัครฟรีที่ github.com

0.2 ตั้งค่า Git ครั้งแรก (ทำครั้งเดียว)

หลังติดตั้ง Git แล้ว เปิด Terminal (Windows ใช้ Git Bash หรือ PowerShell, Mac/Linux ใช้ Terminal) แล้วพิมพ์คำสั่งต่อไปนี้ เพื่อบอก Git ว่าเราคือใคร:

```
git config --global user.name "ชื่อของคุณ"
git config --global user.email "อีเมลที่ใช้สมัคร GitHub"
```

ทำไมต้องทำ? Git จะใช้ชื่อและอีเมลนี้ติดไปกับทุก commit ที่คุณทำ เพื่อให้เพื่อนในทีมรู้ว่าโค้ดบรรทัดไหนใครเขียน และ GitHub จะจับคู่ commit กับบัญชีของคุณอัตโนมัติ

0.3 ตั้งค่า SSH Key หรือ Personal Access Token

ตั้งแต่ปี 2021 เป็นต้นมา GitHub ไม่ให้ใช้ password ปกติเวลา push โค้ดแล้ว ต้องใช้ SSH key หรือ Personal Access Token (PAT) แทน แนะนำให้ใช้ SSH key เพราะตั้งค่าครั้งเดียวจบ ไม่ต้องกรอกรหัสทุกครั้ง

ขั้นตอนสร้าง SSH key (ทำใน Terminal):

```
ssh-keygen -t ed25519 -C "อีเมล@example.com"
# กด Enter รัวๆ ผ่านทุกคำถาม
cat ~/.ssh/id_ed25519.pub
# คัดลอกข้อความที่ขึ้นมาทั้งหมด
```

จากนั้นไปที่ GitHub → คลิกรูป profile มุมขวาบน → Settings → SSH and GPG keys → New SSH key
→ วางข้อความที่คัดลอกมา → ตั้งชื่อ Title อะไรก็ได้ (เช่น “Laptop ที่บ้าน”) → กด Add SSH key

ทำไมต้องทำ? SSH key เปรียบเหมือนกุญแจดิจิทัลที่ผูกกับเครื่องของคุณกับบัญชี GitHub
พอตั้งค่าเสร็จแล้ว เวลา push/pull จะไม่ต้องกรอกรหัสผ่านอีกเลย ปลอดภัยกว่าและสะดวกกว่ามาก

บทที่ 1 สร้าง Repository บน GitHub

Repository (เรียกสั้นๆ ว่า repo) คือ “พื้นที่เก็บโค้ด” กลางบน GitHub ที่ทุกคนในทีมจะส่งโค้ดไปรวมกันที่นี่ คิดเสียว่าเป็น Google Drive แต่สำหรับโค้ดโดยเฉพาะ

1.1 ใครควรเป็นคนสร้าง repo?

ในทีม ให้ “คนเดียวเท่านั้น” เป็นคนสร้าง repo (มักจะเป็นหัวหน้าทีมหรือ DevOps) แล้วจึงเชิญสมาชิกคนอื่นเข้ามา ห้ามให้ทุกคนสร้าง repo ของตัวเอง เพราะจะกลายเป็นคนละโปรเจกต์กัน

1.2 ขั้นตอนการสร้าง (ผ่านหน้าเว็บ GitHub)

6. เข้า github.com แล้ว Sign in
7. คลิกเครื่องหมาย "+" มุมขวาบน → เลือก "New repository"
8. ใส่ Repository name เช่น "team-project-2026" (ใช้ตัวพิมพ์เล็ก คั่นด้วย - ห้ามมีช่องว่างหรือภาษาไทย)
9. ใส่ Description สั้นๆ บอกว่าโปรเจกต์ทำอะไร
10. เลือก "Private" (ถ้าเป็นโปรเจกต์เรียน/ทำงาน) หรือ "Public" (ถ้าจะให้ทุกคนเห็น)
 - **Initialize this repository with:** ตี๊ก Add a README file
 - **.gitignore template:** เลือก "Python" (เดี๋ยวค่อยเพิ่มของ Node.js ทีหลัง)
 - **License:** เลือก MIT License (สำหรับโปรเจกต์เรียนทั่วไป) หรือไม่เลือกก็ได้
11. กด "Create repository" — เป็นอันเสร็จ

ทำไมต้องทำ? การติ๊ก README, .gitignore, License ตั้งแต่แรก จะทำให้ repo มี commit แรกเลย ไม่เป็น repo ว่างๆ ที่ clone มาแล้ว push ลำบาก โดยเฉพาะ .gitignore สำคัญมาก เพราะมันคือไฟล์ที่บอก Git ว่า “อย่าอัปโหลดไฟล์เหล่านี้ละ” เช่น venv, __pycache__, node_modules, .env ซึ่งเป็นไฟล์ส่วนตัวของแต่ละเครื่อง ไม่ควรขึ้น repo

1.3 เชิญเพื่อนเข้าร่วม repo (Collaborator)

12. ในหน้า repo ของคุณ ไปที่แท็บ "Settings"
13. เมนูซ้ายมือ คลิก "Collaborators"

14. กด "Add people" → พิมพ์ username GitHub ของเพื่อน → กด Add

15. เพื่อนจะได้รับอีเมลเชิญ ต้องกดยอมรับก่อนถึงจะมีสิทธิ์ push โค้ด



หมายเหตุ ถ้าเป็น Private repo เพื่อนต้องเป็น Collaborator เท่านั้นถึงจะเห็นและ push ได้
ถ้าเป็น Public repo เพื่อนจะเห็นได้ทุกคนแต่ push ไม่ได้ถ้าไม่ใช่ Collaborator

1.4 ตั้ง Branch Protection (แนะนำมาก)

Branch Protection คือกฎที่ห้ามไม่ให้ใครก็ตาม (รวมถึงเจ้าของ repo เอง) push ตรงเข้า branch main โดยไม่ผ่านการ review จากเพื่อน

16. ไปที่ Settings → Branches → Add branch protection rule

17. Branch name pattern: ใส่ "main"

18. ตี๊ก "Require a pull request before merging"

19. ตี๊ก "Require approvals" (อย่างน้อย 1 คน)

20. กด Create

ทำไมต้องทำ? ถ้าไม่มี protection ใครก็ push เข้า main ได้โดยตรง พอโค้ดเสีย ทั้งทีมพังหมด
การบังคับให้ผ่าน Pull Request ก่อน ทำให้มีคนช่วยตรวจโค้ดอีกชั้น และมีประวัติว่าใคร review
งานของใคร

บทที่ 2 Clone โปรเจกต์มาที่เครื่อง (3 วิธี)

เมื่อ repo ถูกสร้างขึ้นบน GitHub แล้ว สมาชิกทุกคนต้อง “ดาวน์โหลด” repo นี้มาที่เครื่องตัวเอง การ “ดาวน์โหลด” แบบที่ Git ใช้ เรียกว่า Clone (โคลน) ซึ่งจะดึงประวัติทั้งหมดของ repo มาด้วย ไม่ใช่แค่ไฟล์ปัจจุบัน

ทำไมต้องทำ? ถ้าโหลด ZIP ธรรมดา เราจะได้แค่ไฟล์ แต่ไม่ได้ประวัติ commit ทำให้ไม่สามารถ push การเปลี่ยนแปลงกลับขึ้น GitHub ได้ การ clone จึงเป็นวิธี “ที่ถูกต้อง” เสมอเมื่อจะร่วมพัฒนา

2.1 วิธีที่ 1: Clone ผ่าน Terminal (วิธีมาตรฐาน แนะนำที่สุด)

ขั้นตอน

21. เข้าหน้า repo บน GitHub
22. คลิกปุ่มสี่เหลี่ยม "Code" → จะมีแท็บให้เลือก: HTTPS, SSH, GitHub CLI
23. ถ้าตั้งค่า SSH key แล้ว (จากบทที่ 0) เลือกแท็บ SSH
24. คลิกไอคอนคัดลอก (รูปกระดาษ 2 แผ่นซ้อนกัน) ข้างลิงก์ — ลิงก์จะหน้าต่างประมาณ
git@github.com:username/team-project-2026.git
25. เปิด Terminal บนเครื่อง
26. ไปที่โฟลเดอร์ที่อยากเก็บโปรเจกต์ เช่น Desktop หรือ Documents:

```
# Windows (PowerShell)
cd C:\Users\YourName\Documents
```

```
# Mac/Linux
cd ~/Documents
```

27. รันคำสั่ง clone (วางลิงก์ที่คัดลอกมา):

```
git clone git@github.com:username/team-project-2026.git
```

28. Git จะสร้างโฟลเดอร์ใหม่ชื่อเดียวกับ repo จากนั้นเข้าไปในโฟลเดอร์นั้น:

```
cd team-project-2026
```

```
ls      # Mac/Linux
```

```
dir     # Windows
```

จะเห็นไฟล์ README.md, .gitignore ที่เราสร้างไว้ตอนแรก ถือว่า clone สำเร็จ

2.2 วิธีที่ 2: Clone ผ่านปุ่มในหน้าเว็บ (Download ZIP)

วิธีนี้ ไม่แนะนำ สำหรับการทำงานเป็นทีม แต่ขอใส่ไว้เผื่อบางคนงงๆ ว่าทำไมห้ามใช้

29. ในหน้า repo คลิกปุ่ม "Code" สีเขียว
30. คลิก "Download ZIP" ที่ด้านล่างสุด
31. แดก ZIP จะได้ไฟล์เดอริโปรเจกต์

✗ ทำไมไม่แนะนำ? ไฟล์ที่ได้มาไม่มีโฟลเดอร์ .git ซ่อนอยู่ ทำให้ Git ไม่รู้ว่าไฟล์นี้มาจาก repo ไหน จึง push กลับขึ้น GitHub ไม่ได้ ต้องเริ่ม git init ใหม่ ซึ่งจะเสียประวัติทั้งหมดและพอ push จะชนกับ repo ปัจจุบัน

2.3 วิธีที่ 3: Clone ผ่าน VS Code (สำหรับคนที่ไม่ชอบ Terminal)

32. เปิด VS Code
33. กด Ctrl + Shift + P (Mac: Cmd + Shift + P) เพื่อเปิด Command Palette
34. พิมพ์ "Git: Clone" แล้วกด Enter
35. วาง URL ของ repo (คัดลอกจาก GitHub เหมือนวิธีที่ 1)
36. เลือกโฟลเดอร์ที่อยากเก็บ
37. กด "Open" เมื่อ VS Code ถาม

ทำไมต้องทำ? วิธีที่ 3 จริงๆ ก็คือ VS Code รันคำสั่ง git clone ให้แทนเรา ผลลัพธ์เหมือนวิธีที่ 1 ทุกประการ ต่างกันแค่ใช้ UI แทน Terminal เลือกวิธีที่ถนัด แต่แนะนำให้ฝึกใช้ Terminal ไปด้วย เพราะหลายๆ คำสั่ง Git ขั้นสูงทำผ่าน UI ไม่ได้

บทที่ 3 เปิดโปรเจกต์ใน VS Code และเชื่อมต่อ Git

3.1 เปิดโปรเจกต์ใน VS Code

มี 2 วิธีหลัก:

วิธี A: เปิดจาก Terminal (เร็วที่สุด)

```
cd team-project-2026
code .          # จุด "." หมายถึงโฟลเดอร์ปัจจุบัน
```

ถ้าคำสั่ง code ใช้ไม่ได้ ให้เปิด VS Code → กด Ctrl + Shift + P → พิมพ์ "Shell Command: Install 'code' command in PATH" → กด Enter แล้วลองใหม่

วิธี B: เปิดผ่านเมนู

เปิด VS Code → File → Open Folder → เลือกโฟลเดอร์ team-project-2026 → Open

3.2 ติดตั้ง Extension ที่ควรมี

กดไอคอน Extensions ด้านซ้าย (รูปสี่เหลี่ยม 4 ชิ้น) หรือกด Ctrl + Shift + X แล้วค้นหาและติดตั้ง:

- GitLens — ดูประวัติ Git สวยขึ้น เห็นว่าโค้ดบรรทัดไหนใครเขียนวันไหน
- Python (โดย Microsoft) — รัน debug และ format โค้ด Python
- Pylance — type checking สำหรับ Python
- Prettier — จัดรูปแบบโค้ด HTML/CSS/JS ให้สวยอัตโนมัติ
- Live Server — เปิดไฟล์ HTML ใน browser พร้อม auto-reload
- DotENV — แสดงสีให้ไฟล์ .env อ่านง่ายขึ้น

3.3 ตรวจสอบว่า Git เชื่อมกับ VS Code แล้ว

คลิกไอคอน Source Control ด้านซ้าย (รูปสาขาต้นไม้ เลข 3 จุด) ถ้าขึ้นข้อความ "No changes" หรือเห็นชื่อ branch ที่มุมล่างซ้ายของหน้าจอ (ปกติจะขึ้นว่า "main") แปลว่า VS Code เห็น Git แล้ว ไม่ต้องตั้งค่าอะไรเพิ่ม

ทำไมต้องทำ? VS Code มี Git built-in อยู่แล้ว ไม่ต้องลง extension เพิ่ม แค่เปิดโฟลเดอร์ที่ clone มา VS Code จะตรวจเจอโฟลเดอร์ .git ที่ซ่อนอยู่ แล้วทำงานร่วมกับ Git อัตโนมัติ

บทที่ 4 สร้าง Branch ของตัวเอง

4.1 Branch คืออะไร? ทำไมถึงต้องสร้าง?

Branch (สาขา) คือเส้นทางการพัฒนาที่แยกออกจาก main เปรียบเทียบง่ายๆ คือ main เป็นเหมือนต้นไม้หลัก แล้วเราตัดกิ่งย่อยออกมาทดลองทำอะไรก่อน ถ้าทำสำเร็จค่อยเอามาต่อกลับเข้ากิ่งหลัก

4.2 ทำไม ห้าม ทำงานบน branch main โดยตรง?

38. main คือ branch ที่ใช้ deploy ขึ้นเซิร์ฟเวอร์จริง โค้ดที่อยู่ใน main ต้องเป็นโค้ดที่ใช้งานได้ ไม่พัง
39. ถ้าทุกคน push ตรงเข้า main จะ conflict กันตลอด เพราะหลายคนแก้ไฟล์เดียวกัน
40. ถ้าใครเขียนโค้ดผิดแล้ว push ขึ้น main ทันที โค้ดของทุกคนพังตามทั้งทีม
41. ไม่มีโอกาส review โค้ดก่อน เพราะ push ปูบเข้าเลย
42. ย้อน rollback ยากมาก เพราะประวัติปนกัน แยกไม่ออกมาโค้ดส่วนไหนเป็นของพีเจอร์อะไร



กฎทอง main = โค้ดที่ใช้งานจริงเท่านั้น / ทุกการเปลี่ยนแปลง = ทำบน branch ของตัวเอง แล้ว Pull Request เข้า main

4.3 ตั้งชื่อ branch ยังไงให้ดี

รูปแบบที่นิยม: ประเภท/ชื่อ-เรื่อง ทุกอย่างพิมพ์เล็ก คั่นด้วย - ห้ามมีช่องว่างหรือภาษาไทย

ประเภท	ใช้เมื่อไหร่	ตัวอย่าง
feature/	เพิ่มฟีเจอร์ใหม่	feature/login-page
fix/	แก้ bug	fix/login-button-error
docs/	แก้เอกสาร README	docs/update-readme
refactor/	ปรับโครงสร้างโค้ด	refactor/api-routes
chore/	งานเบ็ดเตล็ด เช่น setup	chore/setup-eslint

4.4 ขั้นตอนสร้าง branch (ผ่าน Terminal)

เริ่มต้น ต้องอยู่บน main และต้อง pull โค้ดล่าสุดมาก่อนเสมอ

```
# 1. ตรวจสอบว่าตอนนี้อยู่ branch ไหน
git branch

# * main      <- เครื่องหมาย * คือ branch ที่อยู่

# 2. ดึงโค้ดล่าสุดมาก่อนเสมอ (กันสร้าง branch จากเวอร์ชันเก่า)
git pull origin main

# 3. สร้าง branch ใหม่และย้ายไป branch นั้นเลยในคำสั่งเดียว
git checkout -b feature/my-html-page

# 4. ตรวจสอบอีกครั้ง
git branch

#   main
# * feature/my-html-page
```

ทำไมต้องทำ? คำสั่ง git checkout -b ชื่อใหม่ = "create + switch" สั่งครั้งเดียวเสร็จ ถ้าใช้แค่ git branch ชื่อใหม่ จะสร้างแต่ไม่ย้ายไป ต้อง git checkout ชื่อใหม่ อีกที

4.5 ขั้นตอนสร้าง branch (ผ่าน VS Code)

43. คลิกชื่อ branch ที่มุมล่างซ้ายของ VS Code (ปกติจะเห็นเป็น "main")
44. เลือก "Create new branch..."
45. พิมพ์ชื่อ branch เช่น "feature/my-html-page" แล้วกด Enter
46. VS Code จะสร้างและย้ายไป branch ใหม่ให้อัตโนมัติ มุมล่างซ้ายจะเปลี่ยนชื่อตาม

บทที่ 5 สร้างไฟล์ HTML แล้ว commit & push ขึ้น branch ตัวเอง

ตอนนี้เราอยู่บน branch ของตัวเองแล้ว (feature/my-html-page) เริ่มเขียนโค้ดได้เลย โค้ดที่แก้/สร้างจะอยู่บน branch นี้เท่านั้น ไม่ไปกระทบ main

5.1 สร้างไฟล์ HTML

47. ใน VS Code คลิกขวาที่ explorer ด้านซ้าย → New File

48. ตั้งชื่อ "index.html"

49. พิมพ์โค้ดตัวอย่าง:

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <title>Team Project</title>
</head>
<body>
  <h1>สวัสดีจาก [ชื่อของคุณ]</h1>
  <p>นี่คือไฟล์แรกของฉันบน branch feature/my-html-page</p>
</body>
</html>
```

50. กด Ctrl + S เพื่อบันทึก

5.2 เข้าใจ Workflow ของ Git: 4 ขั้นตอน

Git แบ่งสถานะของไฟล์เป็น 4 ที่อยู่:

- **Working Directory:** โฟลเดอร์ที่เราเห็น เปิดและแก้ไขไฟล์ได้
- **Staging Area:** ห้องรอ — บอก Git ว่า "ไฟล์เหล่านี้พร้อมจะถูกบันทึก"
- **Local Repository:** ประวัติ Git ในเครื่องเรา (ผ่าน commit แล้ว)
- **Remote Repository:** GitHub (ผ่าน push แล้ว)

ขั้นตอนทั้ง 4: edit → add → commit → push

5.3 ดูสถานะปัจจุบัน (git status)

```
git status
```

Git จะบอกว่ามีไฟล์อะไรเปลี่ยน อะไรยังไม่ได้ add ผลลัพธ์จะคล้ายๆ นี้:

On branch feature/my-html-page

Untracked files:

(use "git add <file>..." to include in what will be committed)

index.html

ทำไมต้องทำ? git status คือคำสั่งที่ "ปลอดภัยที่สุด" รันบ่อยๆ ได้ ไม่เปลี่ยนอะไรเลย แค่บอกสถานะ — ใช้บ่อยๆ ก่อนทำคำสั่งอื่นเพื่อกันพลาด

5.4 add ไฟล์เข้า staging area

add เฉพาะไฟล์ที่ต้องการ

```
git add index.html
```

หรือ add ทุกไฟล์ที่เปลี่ยน (ใช้บ่อย แต่ระวังเผลอ add ไฟล์ไม่ตั้งใจ)

```
git add .
```

ทำไมต้องทำ? การ add เป็นเหมือนการ "เลือกของใส่ตะกร้า" ก่อน checkout การแยกเป็นสองขั้นตอน (add แล้ว commit) ช่วยให้เราจัดกลุ่มไฟล์เป็น commit ย่อยๆ ได้ เช่น แก่ 5 ไฟล์ แต่ commit แยกเป็น 2 ก่อนตามฟีเจอร์

5.5 commit (บันทึกประวัติในเครื่อง)

```
git commit -m "feat: add initial index.html with greeting"
```

ข้อความหลัง -m เรียกว่า commit message ควรเขียนสั้นๆ บอกว่าเปลี่ยนอะไร รูปแบบที่นิยม (Conventional Commits):

- feat: เพิ่มฟีเจอร์ใหม่ — เช่น "feat: add login page"
- fix: แก้ bug — เช่น "fix: button not responding on mobile"
- docs: แก้เอกสาร

- style: จัด format โค้ด ไม่ได้แก้ logic
- refactor: ปรับโครงสร้างโค้ด ไม่ได้เปลี่ยน behavior
- test: เพิ่ม/แก้ test
- chore: งานเบ็ดเตล็ด เช่น setup, config

ทำไมต้องทำ? commit message ที่ดีทำให้ทีมรู้ทันทีว่าแต่ละ commit ทำอะไร โดยไม่ต้องเปิดดูโค้ด ห้ามใช้ message มั่วๆ เช่น "update", "fix", "asdf"

5.6 push ขึ้น GitHub

ครั้งแรกที่ push branch นี้ ต้องใส่ -u (upstream)

```
git push -u origin feature/my-html-page
```

ครั้งถัดไป push ง่ายๆ ได้เลย

```
git push
```

ทำไมต้องทำ? -u origin feature/my-html-page จะบอก Git ว่า branch ในเครื่องนี้ผูกกับ branch ชื่อเดียวกันบน GitHub ตั้งแต่นี้ไป ใช้แค่ git push / git pull ง่ายๆ Git จะรู้เองว่าต้อง push/pull ไปที่ไหน

5.7 ตรวจสอบบน GitHub

เข้า GitHub → หน้า repo → ด้านบนซ้ายจะมี dropdown ที่เขียนว่า "main" คลิกแล้วจะเห็น branch ของเราโผล่ขึ้นมา เลือกเพื่อดูไฟล์บน branch นี้

 **Workflow สรุปสั้น** edit → git add . → git commit -m "..." → git push (ทำซ้ำๆ ทุกครั้งที่แก้โค้ด)

บทที่ 6 สร้างไฟล์ .env สำหรับรัน Python

ไฟล์ .env (อ่านว่า "ดอท-อี-เอ็น-วี") คือไฟล์เก็บค่าตั้งค่าที่ "เป็นความลับ" หรือ "เปลี่ยนตามเครื่อง" เช่น รหัสผ่านฐานข้อมูล, API key, ที่อยู่เซิร์ฟเวอร์

6.1 ทำไม .env ห้ามขึ้น GitHub ได้เด็ดขาด

51. ในนั้นมีรหัสผ่าน/API key ถ้าเผลอ push ขึ้น public repo คนทั้งโลกเห็นได้
52. ค่าใน .env ต่างกันในแต่ละเครื่อง (เครื่องเรา DB อยู่ที่ localhost เครื่องเพื่ออาจอยู่อื่น)
53. ค่าใน production (เซิร์ฟเวอร์จริง) ต้องไม่เหมือนกับ development

 **เรื่องจริงที่เกิดขึ้น** บริษัทใหญ่ๆ หลายแห่งโดน hack เพราะพนักงานเผลอ push ไฟล์ .env ที่มี AWS access key ขึ้น GitHub แม้แค่ 5 นาทีก็โดน bot scan เจอแล้ว ถูกใช้ขุดเหรียญหรือลบข้อมูลจนเสียหายหลายล้าน

6.2 ตรวจสอบให้แน่ใจว่า .env อยู่ใน .gitignore

เปิดไฟล์ .gitignore (อยู่ที่ root ของ project ที่เราสร้างตอนทำ repo) แล้วตรวจให้แน่ใจว่ามีบรรทัดเหล่านี้:

```
# Environment
.env
.env.local
.env*.local

# Python
__pycache__/*
*.pyc
venv/
.venv/

# Node
node_modules/
```

```
# OS
.DS_Store
Thumbs.db
```

```
# IDE
.vscode/
.idea/
```

ทำไมต้องทำ? ไฟล์ที่ระบุใน .gitignore จะถูก Git เพิกเฉย — แม้เราจะ git add . ก็จะไม่ถูก add เข้า staging นี่คือ safety net ชั้นแรก

6.3 สร้างไฟล์ .env.example (ตัวอย่าง — ขึ้น GitHub ได้)

เนื่องจาก .env จริงจะไม่ขึ้น GitHub เพื่อนใหม่ที่ clone repo มาจะไม่รู้ว่าต้องสร้าง .env หน้าตาแบบไหน เราจึงสร้างไฟล์ตัวอย่างชื่อ .env.example ที่บอก "โครงสร้าง" แต่ไม่มีค่าจริง

สร้างไฟล์ .env.example ใน root ของโปรเจกต์:

```
# Database
DB_HOST=localhost
DB_PORT=5432
DB_NAME=team_project_db
DB_USER=your_username_here
DB_PASSWORD=your_password_here

# API Keys
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxx
SECRET_KEY=change_me_to_random_string

# App Config
DEBUG=True
APP_PORT=8000
```


ทำไมต้องทำ? .env.example ทำหน้าที่เป็น “เทมเพลต” ให้คนอื่น และเป็น “เอกสารกึ่งทางการ” ว่าโปรเจกต์นี้ต้องการ environment variable อะไรบ้าง ทุกครั้งที่เพิ่ม variable ใหม่ใน .env อย่าลืมเพิ่มใน .env.example ด้วย (แต่ไม่ใช่ค่าจริง)

6.4 สร้างไฟล์ .env ของตัวเอง (ในเครื่องเท่านั้น)

```
# คัดลอกจากตัวอย่าง
cp .env.example .env      # Mac/Linux
copy .env.example .env    # Windows
```

จากนั้นเปิดไฟล์ .env แล้วใส่ค่าจริงของตัวเอง

6.5 ตรวจสอบว่า .env ไม่หลุดเข้า Git

```
git status
# ต้อง "ไม่เห็น" .env ในรายการเลย
# ถ้าเห็น แสดงว่า .gitignore ตั้งค่าไม่ถูก ห้าม commit ได้ขาด
```

6.6 รัน Python ด้วยค่าใน .env

ติดตั้ง library python-dotenv ก่อน:

```
# สร้าง virtual environment (แยกของแต่ละ project)
python -m venv venv

# เปิดใช้งาน venv
source venv/bin/activate  # Mac/Linux
venv\Scripts\activate     # Windows

# ติดตั้ง
pip install python-dotenv

# บันทึก dependencies
pip freeze > requirements.txt
```

ตัวอย่างโค้ด Python ที่อ่าน .env (สร้างไฟล์ชื่อ app.py):

```
import os
from dotenv import load_dotenv

# โหลดค่าจากไฟล์ .env เข้าสู่ environment variables
load_dotenv()

DB_HOST = os.getenv("DB_HOST")
DB_NAME = os.getenv("DB_NAME")
DB_USER = os.getenv("DB_USER")
DB_PASS = os.getenv("DB_PASSWORD")

print(f"Connecting to {DB_NAME} at {DB_HOST} as {DB_USER}")
# password ใช้ภายในเท่านั้น ไม่ควร print
```

รันด้วย:

```
python app.py
```

6.7 commit & push (แต่จะ push เฉพาะ .env.example ไม่ใช่ .env)

```
git status
# ควรเห็น: .env.example, .gitignore, app.py, requirements.txt
# ไม่ควรเห็น: .env, venv/

git add .env.example .gitignore app.py requirements.txt
git commit -m "chore: add env example and python dotenv setup"
git push
```



ถ้าเผลอ commit .env ไปแล้ว ลบจาก Git history ทันที: `git rm --cached .env` แล้ว commit ใหม่ / หลังจากนั้น ถ้าวารรหัสผ่านที่อยู่ในไฟล์ "รั่ว" แล้วต้อง rotate ทันที (เปลี่ยนรหัส DB, สร้าง API key ใหม่) เพราะประวัติ Git ลบยาก

บทที่ 7 เปิด Pull Request เพื่อรวมงานกลับไป main

Pull Request (PR) คือการ “ขออนุญาต” เอาโค้ดจาก branch ของเรา รวมเข้ากับ main เป็นโอกาสให้เพื่อนช่วยอ่านโค้ด ทักท้วง และยืนยันว่าโค้ดพร้อมรวม

7.1 ขั้นตอนเปิด PR

54. push branch ของเราขึ้น GitHub ให้เรียบร้อยก่อน (ดูบทที่ 5)
55. เข้าหน้า repo บน GitHub
56. GitHub จะขึ้นกล่องสี่เหลี่ยมว่า "feature/my-html-page had recent pushes" → คลิก "Compare & pull request"
57. หรือถ้าหากไม่เห็นกล่อง: คลิกแท็บ "Pull requests" → "New pull request" → เลือก base: main, compare: feature/my-html-page
58. ตั้ง title สั้นๆ บอกว่าทำอะไร เช่น "Add initial HTML page and Python env setup"
59. ในช่อง description เขียนรายละเอียดว่าเปลี่ยนอะไรบ้าง, ทดสอบยังไง, มีอะไรที่อยากให้ reviewer สนใจเป็นพิเศษ
60. ทางขวามี Reviewers — เลือกเพื่อนในทีมที่อยากให้ review
61. กด "Create pull request"

7.2 รอ review และตอบกลับ

เพื่อนจะอ่านโค้ดและอาจ comment ขอแก้ไข ถ้าต้องแก้ไข:

```
# แก้โค้ดในเครื่อง
git add .
git commit -m "fix: address review comments"
git push
```

Commit ใหม่จะถูกเพิ่มเข้า PR เดิมโดยอัตโนมัติ ไม่ต้องเปิด PR ใหม่

7.3 Merge เข้า main

เมื่อ reviewer Approve แล้ว:

62. กลับมาที่หน้า PR → กดปุ่ม "Merge pull request" → "Confirm merge"

63. GitHub จะถามว่าจะลบ branch ทิ้งไหม → กด "Delete branch" ได้เลย (ในเครื่องยังอยู่
เฉพาะบน GitHub)

ทำไมต้องทำ? ลบ branch ที่ merge แล้วทำให้รายชื่อ branch บน GitHub สะอาด ไม่รก โค้ดยังอยู่ใน
history เสมอ ไม่หาย

บทที่ 8 อัปเดต branch ของตัวเองเมื่อเพื่อน merge งานเข้า main

ปัญหาที่พบบ่อย: เพื่อน merge งานเข้า main แล้ว แต่ branch ของเรายังเป็นเวอร์ชันเก่า ทำให้พอจะ merge บ้างจะเกิด conflict วิธีแก้คือ sync main ใหม่ใส่ branch ตัวเอง

8.1 ขั้นตอน

```
# 1. กลับไป main ก่อน
git checkout main

# 2. ดึงโค้ดล่าสุดจาก GitHub
git pull origin main

# 3. กลับมา branch ตัวเอง
git checkout feature/my-html-page

# 4. ดึง main มารวมเข้า branch ตัวเอง
git merge main

# 5. ถ้ามี conflict → แก้ในไฟล์ที่ Git บอก → add → commit
# ถ้าไม่มี conflict → merge เสร็จเลย

# 6. push
git push
```

8.2 ทำบ่อยแค่ไหน?

แนะนำให้ทำ "ทุกเข้าก่อนเริ่มงาน" หรือ "ทุกครั้งที่เพื่อนแจ้งว่า merge อะไรเข้า main แล้ว"

ทำไมต้องทำ? ยิ่ง branch ของเรา “ตามหลัง” main มาก ยิ่งมีโอกาส conflict สูง การ sync ถี่ๆ ช่วยให้ conflict (ถ้ามี) มีขนาดเล็ก แก้ง่าย เทียบกับการสะสมไว้แก้ทีเดียวซึ่งอาจวุ่นวายมาก

บทที่ 9 คำสั่ง Git ที่ใช้บ่อย (Cheat Sheet)

กลุ่ม	คำสั่ง	ทำอะไร
ดู	git status	ดูสถานะไฟล์ปัจจุบัน
ดู	git log --oneline	ดูประวัติ commit สั้นๆ
ดู	git branch	ดู branch ทั้งหมดในเครื่อง
ดู	git branch -a	ดู branch ทั้งหมด (รวม remote)
ดู	git diff	ดูว่าไฟล์ที่แก้ มีอะไรเปลี่ยนแปลง
Branch	git checkout -b ชื่อ	สร้าง branch ใหม่และย้ายไป
Branch	git checkout ชื่อ	ย้ายไป branch ที่มีอยู่
Branch	git branch -d ชื่อ	ลบ branch ในเครื่อง
Branch	git push origin --delete ชื่อ	ลบ branch บน GitHub
บันทึก	git add .	add ทุกไฟล์ที่เปลี่ยนแปลง
บันทึก	git commit -m "..."	บันทึก commit ในเครื่อง
บันทึก	git commit --amend	แก้ commit ล่าสุด (ยังไม่ push)
ซิงค์	git pull	ดึงโค้ดล่าสุดจาก GitHub
ซิงค์	git push	ส่งโค้ดของเราขึ้น GitHub
ซิงค์	git fetch	ดึงข้อมูลโดยยังไม่ merge
ย้อน	git restore ไฟล์	ย้อนไฟล์กลับเป็นก่อนแก้ (ระวัง!)
ย้อน	git reset --soft HEAD~1	ยกเลิก commit ล่าสุด แต่เก็บโค้ดไว้
ย้อน	git revert <commit>	สร้าง commit ที่ย้อน commit เก่า
ฉุกเฉิน	git stash	พักโค้ดที่ยังไม่ commit ไว้ก่อน
ฉุกเฉิน	git stash pop	เรียกโค้ดที่พักไว้กลับมา

บทที่ 10 ปัญหาที่พบบ่อยและวิธีแก้

10.1 "error: Your local changes would be overwritten by merge"

สาเหตุ: คุณแก้ไขไฟล์แต่ยังไม่ commit แล้วไป pull

แก้: เลือกทางใดทางหนึ่ง

```
# วิธี A: commit ก่อน แล้วค่อย pull
git add .
git commit -m "WIP: save before pull"
git pull
```

```
# วิธี B: พักโค้ดไว้ก่อน
git stash
git pull
git stash pop
```

10.2 Merge conflict (โค้ดชนกัน)

Git จะใส่เครื่องหมายไว้ในไฟล์:

```
<<<<<< HEAD
โค้ดของเรา
=====
โค้ดของเพื่อน (จาก main)
>>>>>> main
```

วิธีแก้: เปิดไฟล์ ลบเครื่องหมายทิ้ง เก็บโค้ดที่ถูกต้องไว้ บางครั้งต้องเอาทั้งสองส่วนผสมกัน จากนั้น:

```
git add ไฟล์ที่แก้แล้ว
git commit # message จะถูกเติมให้อัตโนมัติ
```

10.3 push แล้วโดน reject (Updates were rejected)

สาเหตุ: เพื่อน push อะไรบางอย่างขึ้น branch เดียวกันก่อนเรา

```
git pull --rebase
```



```
# แก้ conflict ถ้ามี  
git push
```

10.4 เผลอ commit ผิด branch

```
# ถ้ายังไม่ push  
git log --oneline # คัดลอก commit hash  
git reset HEAD~1 # ย้อน commit (โค้ดยังอยู่)  
git stash # พักไว้  
git checkout branch-ที่-ถูก  
git stash pop  
git add .  
git commit -m "..."
```

10.5 อยากเปลี่ยน commit message ที่เพิ่งทำไป

```
# ถ้ายังไม่ push  
git commit --amend -m "ข้อความใหม่"  
  
# ถ้า push ไปแล้ว (อันตราย ระวังกระทบเพื่อน)  
git commit --amend -m "ข้อความใหม่"  
git push --force-with-lease
```

10.6 อยากดูว่าใครแก้บรรทัดไหน

```
git blame ชื่อไฟล์  
# หรือใช้ extension GitLens ใน VS Code ดูได้สวยงามกว่า
```

10.7 อยากย้อนกลับไปดูโค้ดเวอร์ชันเก่า

```
git log --oneline  
# คัดลอก commit hash เช่น a1b2c3d  
  
git checkout a1b2c3d
```

```
# ดูโค้ด ณ จุดนั้น (อย่าแก้ไขที่นี่)
```

```
git checkout main    # กลับมา branch หลัก
```

10.8 ลบไฟล์ที่เผลอ commit ไปแล้ว

```
# ลบทั้งไฟล์จริงและจาก Git
```

```
git rm ชื่อไฟล์
```

```
git commit -m "remove unused file"
```

```
# ลบจาก Git แต่เก็บไฟล์จริงไว้ในเครื่อง (เช่น .env ที่เผลอ commit)
```

```
git rm --cached ชื่อไฟล์
```

```
git commit -m "stop tracking sensitive file"
```

สรุปและเช็คลิสต์ก่อนเริ่มงานทุกวัน

ทุกครั้งก่อนเริ่มเขียนโค้ดในวันใหม่ ให้ทำตามนี้:

64. git checkout main — กลับมา branch หลัก
65. git pull origin main — ดึงงานล่าสุดของทีม
66. git checkout feature/ของคุณ — ย้ายกลับมา branch ตัวเอง
67. git merge main — เอางานล่าสุดของทีมมารวม
68. เริ่มเขียนโค้ด
69. ระหว่างทาง: edit → git add . → git commit -m "..."
70. สิ้นวัน: git push — ส่งงานขึ้น GitHub

เมื่อฟีเจอร์เสร็จสมบูรณ์: เปิด Pull Request → รอ review → merge → ลบ branch → ทำฟีเจอร์ต่อไปบน branch ใหม่

 **Key Takeaways** (1) main คือพื้นที่ศักดิ์สิทธิ์ ห้ามแตะตรงๆ / (2) ทำงานบน branch ของตัวเองเสมอ / (3) commit บ่อยๆ พร้อม message ที่อ่านเข้าใจ / (4) push ทุกวันเพื่อ backup / (5) .env ไม่มีวันขึ้น GitHub — ใช้ .env.example แทน / (6) sync main เข้า branch ตัวเองทุกวันก่อนทำงาน

— จบคู่มือ —

ขอให้สนุกกับการทำโปรเจกต์เป็นทีม! 